Sistemi operativi II

Anno 2021

Valentino Bocchetti

Contents

1 Introduzione al corso **2**

2 Lezione del 08-03 **3**

2.1 I sistemi operativi 3

2.2 Sistemi mono e multi programmati 3

2.3 Sistemi paralleli 4

2.4 Sistemi distribuiti 4

2.5 Ambiente di elaborazione (Client-Server o Peer-to-Peer) 4

2.6 Cloud computing 5

2.7 Sistemi mobile 5

2.8 Avvio di un sistema operativo nel dettaglio 5

2.9 Tabella degli interrupt 6

2.10 Memoria centrale 6

2.11 Memoria cache 6

2.12 Memoria di massa 6

2.13 I processi 7

2.14 Activity 8

2.15 Politiche utilizzate dallo scheduler 9

2.16 Sistema mono e multi tasking 9

2.17 Throughput (utilizzo della CPU) 10

2.18 Overhead 10

3 Lezione del 12-03 **10**

3.1 Sincronizzazione dei processi 10

3.2 Comunicazione con i dispositivi 12

3.3 File system 12

3.4 La shell dei comandi 13

3.5 La memoria virtuale 13

3.6 Memoria virtuale in Linux 14

3.7 Spazio di indirizzi di un processo 15

1 Introduzione al corso

`Ricevimento` (da concordare almeno 24 ore prima) → Venerdì 10-12 (chat privata su teams)

`Email` → silviobarra@unina.it con oggetto **[INFO-S02]** (iscritto al *anno di corso* e *corso di laurea* SO2 esame facoltativo o obbligatorio)

Obiettivi

- Scelte progettuali ed implementative di un reale sistema operativo a base linux
- Sistemi distribuiti e del cloud computing
- Sistemi operativi ad uso mobile
- IoT

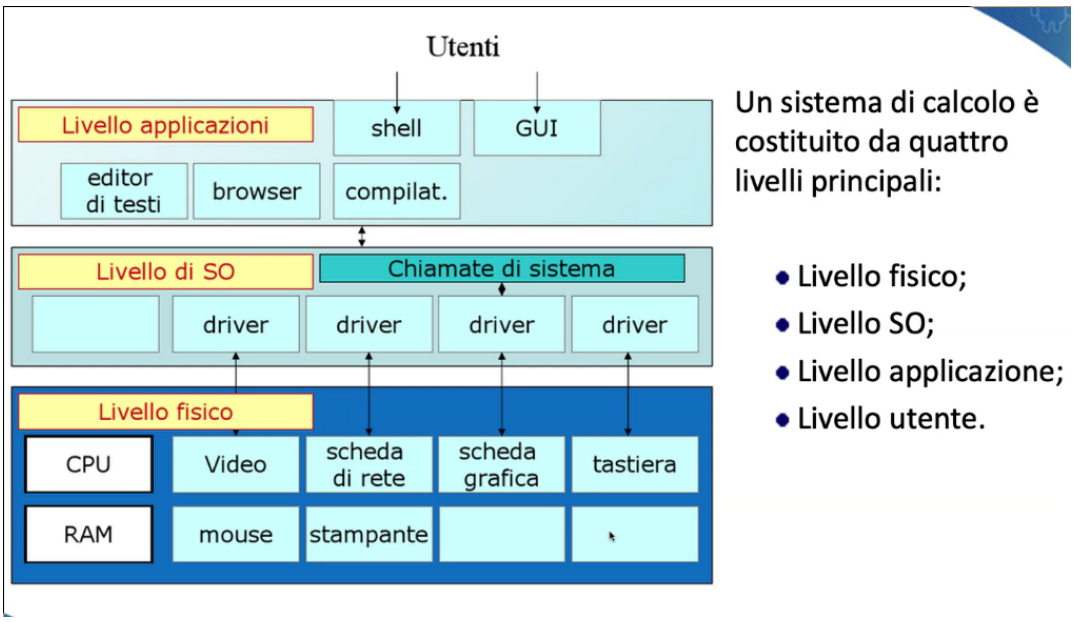
Prove intercorso:

- Domande sul sistema linux
- Domande sui sistemi mobile

2 Lezione del 08-03

2.1 I sistemi operativi

Le componenti di un sistema di calcolo è così definita:



Un **sistema operativo** è difficile da definire (in base alle grandi variazioni che presenta). Una definizione è quella di definirlo come un insieme di software atti a gestire l'hardware sottostante.

Può essere visto come un allocatore di risorse:

- memoria;
- dispositiv di `I/O`;
- processi

Può essere visto come un programma di controllo:

- gestore dei programmi di sistema ed applicativi;
- verifica del corretto funzionamento dei dispositivo di `I/O`

2.2 Sistemi mono e multi programmati

Un sistema è detto **mono-programmato** quando in ogni momento è presente una sola **immagine** (pacchetto processo + dati)

Con l'introduzione del concetto di **spooling**, nascono i primi sistemi **multi-programmati** (il sistema è in grado di eseguire più job in concorrenza). Ciò comporta di tener conto della gestione delle risorse ad essi dedicate:

- Tempo di calcolo
- Memoria
- `I/O`

In questi sistemi si ha il aso di sistemi multi-utenti. Di conseguenza nascono i sistemi **time-sharing** (nascita dei primi PC). Ogni processo utilizza periodicamente un intervallo di tempo prestabilito (un **quanto**)

2.3 Sistemi paralleli

Si arriva alla nascita dei sistemi **Paralleli** dotati di più CPU :

- Symmetric multiprocessing (SMP) $\rightarrow$ ogni processore esegue una copia identica del SO
- Asymmetric multiprocessing (AMP) $\rightarrow$ a ciascun processore è assegnato un task specifico; il processore **master** schedua e allora il lavoro per i processori **slave**

2.4 Sistemi distribuiti

Ripartiscono l'elaborazione su diversi processori. Ciascun processore ha una memoria locale e i processori comunicano con gli altri attraverso di comunicazione quali bus ad alta velocità o rete. La rete può essere:

- Locale (LAN);
- Geografica (WAN);
- Metropolitana (MAN);
- Small (SAN).

2.5 Ambiente di elaborazione (Client-Server o Peer-to-Peer)

Esistono due principali tipi di server:

- server per l'elaborazione;
- file server

Nei sistemi Peer-to-Peer invece ciascun pc può essere server o client

- Il carico è suddiviso su più server (si evitano i sovraccarichi);
- Necessita una politica di gestione
- Nuova entità (**load balancer**)

2.6 Cloud computing

Paradigma di uso delle risorse computazionali attraverso il cloud (memorizzazione, archiviazione ed elaborazione dati)

2.7 Sistemi mobile

Il sistema operativo in questo caso ha orientamento specifico (a causa delle dimensioni del dispositivo)

2.8 Avvio di un sistema operativo nel dettaglio

All'avvio (o al reset) di un sistema il programma di **bootstrap** presente sul firmware inizializza i componenti del sistema, carica ed avvia il SO, che rimane in attesa di:

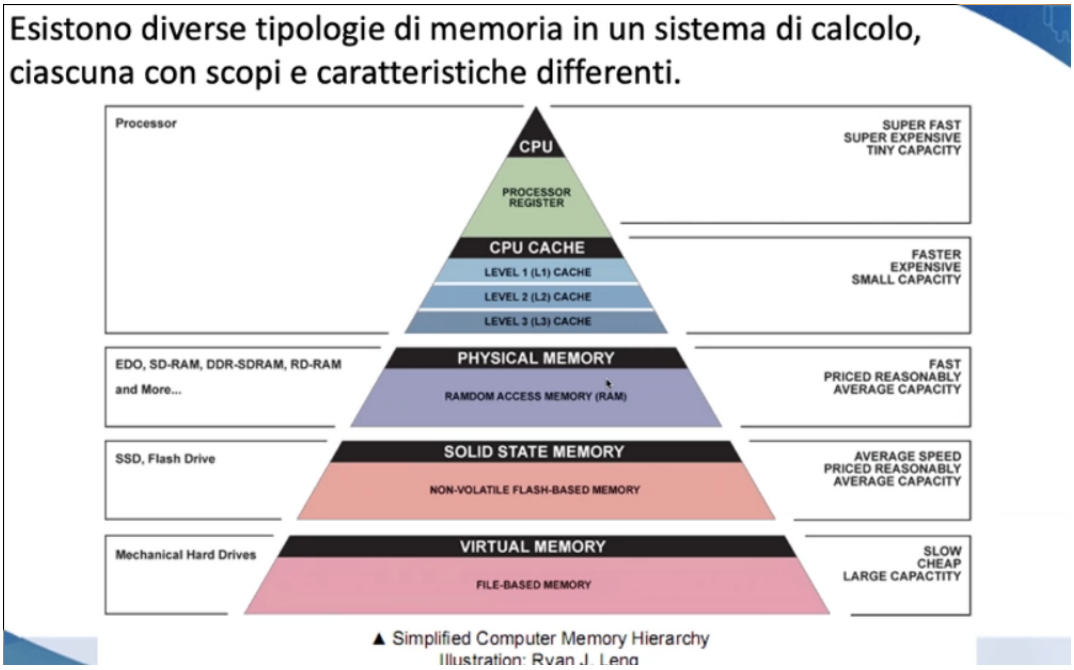
- Un programma applicativo da eseguire;
- Un interrupt → segnale generato da un controller di dispositivo o da un programma e invaito direttamente alla CPU, tramite bus;
- una exception → segnale generato da un programma a seguito di una condizione eccezionale di errore o di una specifica richiesta tramite system call.

La sequenza di avvio è:

Accensione → POST → controllo hard disk e Plug & Play → Avvio del bootloader → Avvio del SO

2.9 Tabella degli interrupt

Gli interrupt vengono gestiti mediante una strategia basata su una tabella



2.10 Memoria centrale

È rappresentata ccome un vettore di parole machina dalle centinaia di migliaia all'ordine dei miliardi di parole. Nella memoria centrale vengono caricati programmi e i dati

2.11 Memoria cache

Memoria molto veloce con lo scopo di ridurre al minimo gli accessi ad unità di memoria più lente. Bisogna garantire la coerenza delle informazioni memorizzate in una o più unità di memoria cache (il cui grado di complessità aumenta a seconda del tipo di sistema)

2.12 Memoria di massa

Divise in secondarie (dischi magnetici, solidi) e terziarie (dischi ottici, CD, DVD, nastri). Le responsabilità del SO rispetto alla gestione delle memorie di massa sono:

- gestione dello spazio libero;
- assegnazione dello spazio;
- scheduling del disco.

I dati vengono scritti nelle memoria mediante file

2.13 I processi

Un processo può essere definito come un programma di elaborazione o **job**. Un processo è un entità attiva, dotata di un program counter. La sua esecuzione è sequenziale e necessita di determinate risorse:

- CPU
- memoria
- dispositivi `I/O`;
- file

Il SO deve garantire alcune operazioni fondamentali per i processi:

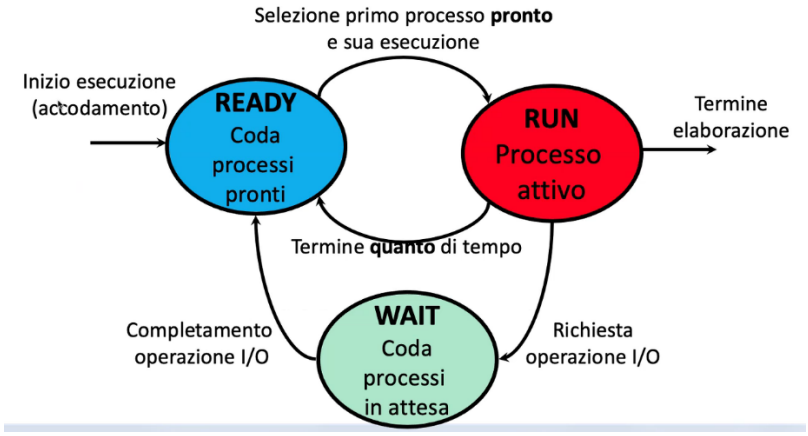
- creazione e cancellazione di processi;
- sospensione e ripristino di processi;
- sincronizzazione di processori;
- comunicazione fra processi;
- soluzioni per lo stato di **deadlock** di processi (questo compito spetta al kernel attraverso lo **scheduler**)

Gli stati di un processo

Ci si può trovare in 2 stati:

- attesa;
- pronto;

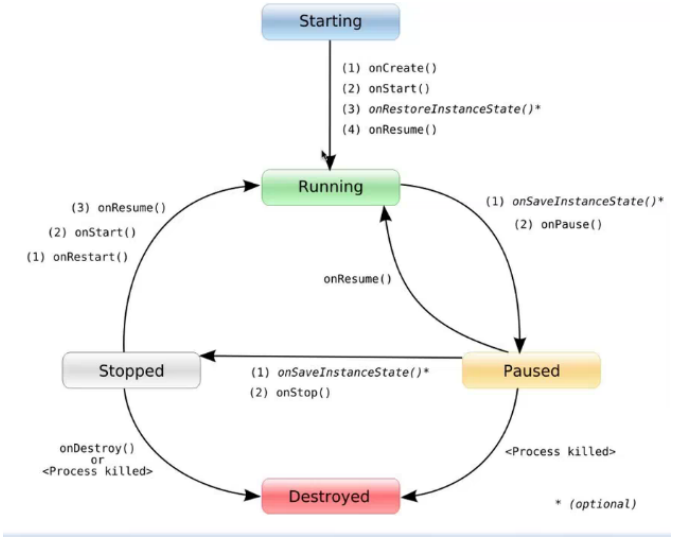
In ogni instante di tempo vi è un solo stato attivo



2.14 Activity

Una **Activity** è caratterizzata da un ciclo di vita ed in ogni istante si trova in uno dei seguenti stati:

- `Active/Resumed` (running) → essa è visibile e può ricevere input dall'utente;
- Paused → essa è parzialmente visibile a causa di un'altra activity in foreground, non può ricevere input dall'utente (*es* è in attesa a causa di un alert dialog presente al di sopra di essa);
- Stopped → essa non è visibile (resta in background)

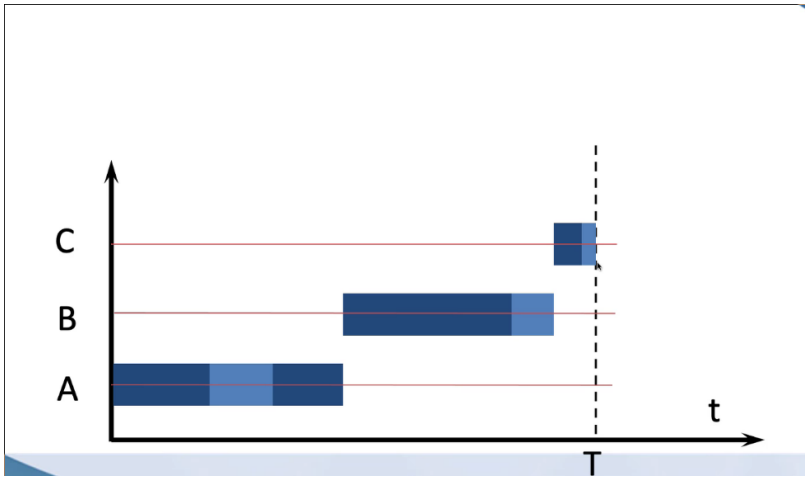


2.15 Politiche utilizzate dallo scheduler

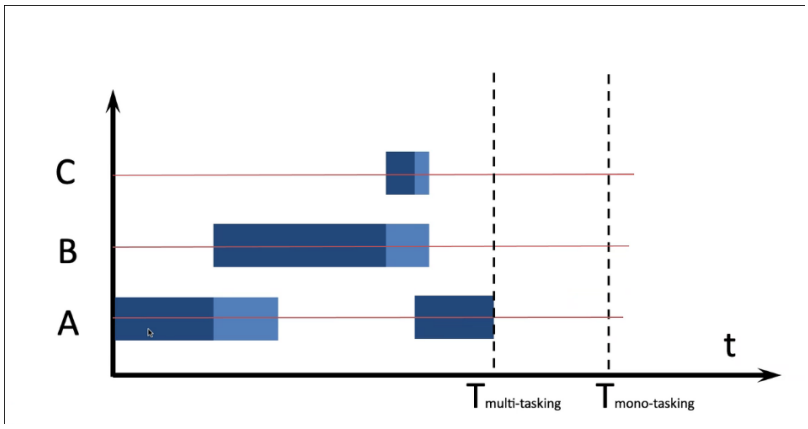
- **Preemtive** → la CPU in uso può essere tolta e passata a un altro in qualsiasi momento;
- **Non Preemtive** → una volta che il processo ha ottenuto l'us della CPU non può essere interrotta fino a che lui stesso non la rilascia;

2.16 Sistema mono e multi tasking

La gestione della CPU (mantenere in maniera efficiente il suo uso) è un obiettivo del sistema operativo. Nel caso di un sistema mono-tasking avremo:



Nel caso di un sistema multi-tasking avremo:



(*In blu scuro il tempo di utilizzo CPU e in chiaro il tempo di attesa di eventi esterni*)

2.17 Throughput (utilizzo della CPU)

Si ha dal rapporto di Tp/Tt dove:

- Tp = tempo dedicato dalla CPU alla esecuzione dei processi utente;
- Tt = tempo totale di osservazione

Si ha un'alta efficienza con un throughput tendente ad 1

2.18 Overhead

Tempo impiegato dal SO per trasferire il controllo da un programma ad un altro

3 Lezione del 12-03

3.1 Sincronizzazione dei processi

Uno dei compiti che spetta al SO è la sincronizzazione dei processi. Più processi su una macchina possono essere:

- In **competizione**
  - se si cerca di usare la stessa risorsa contemporaneamente;
- In **cooperazione**
  - quando uno ha bisogno dell'altro per evolvere;

La sincronizzazione avviene grazie a variabili condivise dette **semafori** o a scambio di messaggi tra processi

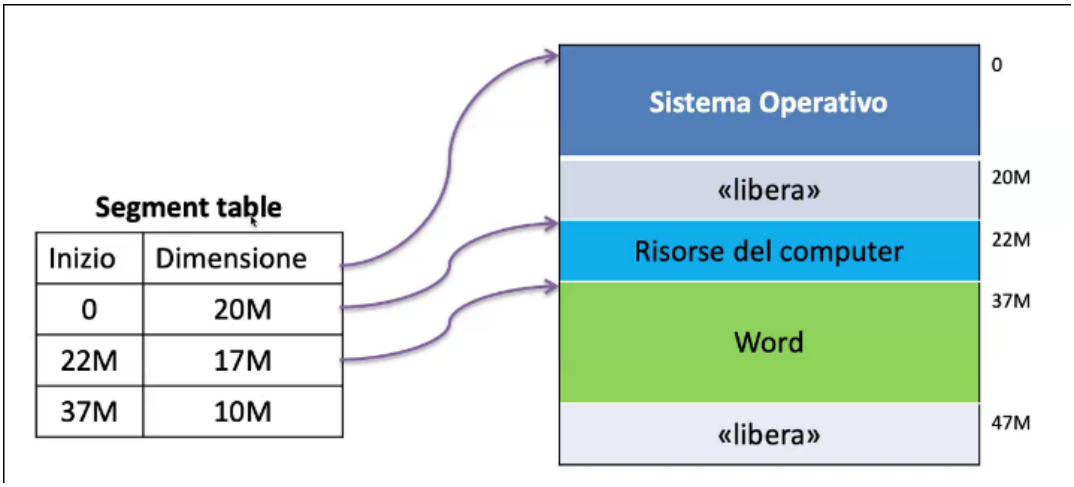
Ogni processo per essere eseguito deve sessere caricato in memoria. Nei sistemi multitasking permette di caricare più processi contemporaneamente.

Il gestore della memoria deve:

- Tenere traccia di quali parti della memoria sono usate e da chi;
- Decidere quali processi caricare quando diventa disponibile spazio in memoria;
- Allocare e deallocare memoria.

Per ogni blocco di memoria il SO mantiene un **descrittore** con le informazioni essenziali. La memoria fisica è suddivisa in un determinato numero di aeree (segmenti)

Ogni entry nella **segment table** contiene l'indirizzo di base di un segmento e la sua lunghezza



Lo svantaggio principale è la frammentazione (la memoria non viene sfruttata completamente). Per ovviare a ciò esiste un processo di compattamento → traslazione dei programmi.

Politiche di gestione dei frammenti

- **First-fit** → alloca il primo frammento libero sufficiente;
- **Best-fit** → alloca il più piccolo frammento libero sufficiente;
  - ricerca sull'intera lista e produce frammenti piccoli
- **Worst-fit** → alloca il più grande frammento
  - ricerca sull'intera lista e produce frammenti grandi

Nei moderni SO si utilizza la **paginazione dei segmenti**, ossia un segmento viene realizzato tramite un insieme di pagine

La suddivisione della memoria centrale in blocchi di dimensioni fisse (pagine) tipicamente 2KB o 4KB.

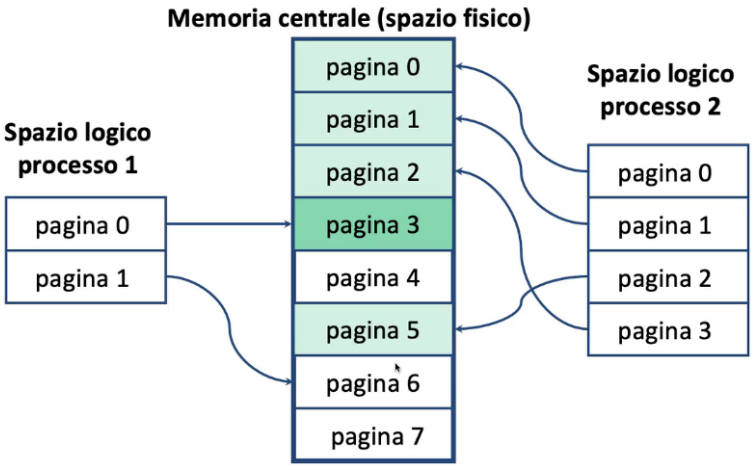
L'immagine di un processo viene suddivisa in pagine caricate all'interno della memoria principale in modo indipendente, anche in posizioni non contigue.

Il SO gestisce una **Page Table** che memorizza per ogni pagina (spazio logico) del processo e la sua posizione in memoria centrale (spazio fisico)

Frammentazione interna ed esterna

- **Frammentazione interna** → Con i blocchi di dimensioni fisse, all'interno di ogni blocco ci sarà uno spazio non utilizzato

- **Frammentazione esterna** → con i blocchi di dimensioni variabili, dopo un certo numero di allocazione e deallocazione di processi, si formeranno un certo numero di porzioni di memoria libere (buchi) di dimensioni insufficienti a contenere un processo



Se durante l'esecuzione del processo, la CPU fa riferimento ad un indirizzo che appartiene ad una pagina non caricata in memoria centrale (page fault), il SO provvede a mettere il processo in stato d'attesa e a far partire il caricamento della pagina richiesta (swap), dopodichè il processo può riprendere la sua esecuzione.

Nella paginazione esiste ancora il problema della frammentazione, che è però limitato al fatto che l'ultima pagina di ogni processo è solo parzialmente occupata. Mediamente si può considerare uno spreco pari a circa 1/2 pagina per ogni processo

3.2 Comunicazione con i dispositivi

Il gestore delle periferiche permette la comunicazione con le varie periferiche. Particolari programmi detti device driver gestiscono le operazioni di `in/out` tra le varie periferiche collegate alla macchina.

Le funzioni svolte dai device driver sono:

- Rendere trasparenti le caratteristiche fisiche tipiche di ogni dispositivo;
- Gestire la comunicazione dei segnali verso i dispositivi;
- Gestire i confiltti, nel caso in cui due o più task vogliono accedere contemporaneamente allo stesso dispositivo

3.3 File system

Gestisce le memorie di massa (organizza logicamente i dati e le possibili operazioni su di essi)

3.4 La shell dei comandi

L'interprete dei comandi o shell è quella parte più esterna di un SO che riceve ed elabora le istruzioni impartite da un utente Il SO presenta due tipi di intefaccia:

- testuale;
- grafica (diversa tra mobile e pc);

3.5 La memoria virtuale

Sistema `SW/HW` in grado di simulare uno spazio di memoria centrale maggiore di quello fisicamente presente

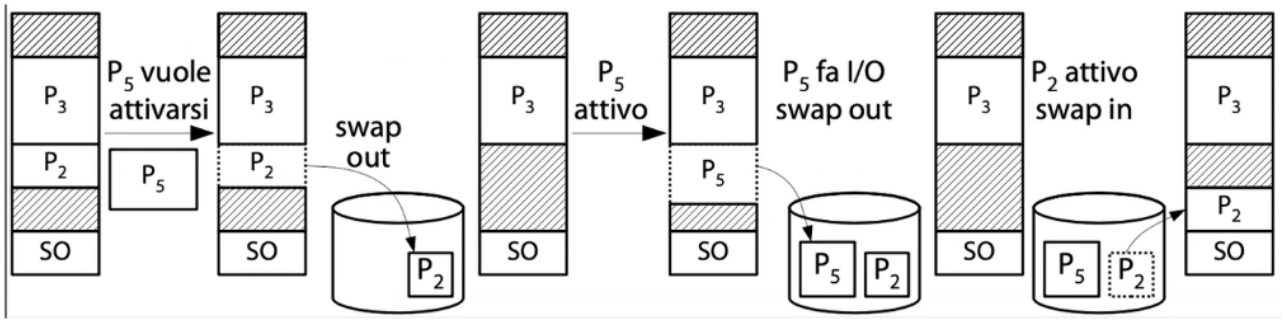
Rappresenta in modo astratto e logico la memoria cetrale di un calcolatore

Memory management unit

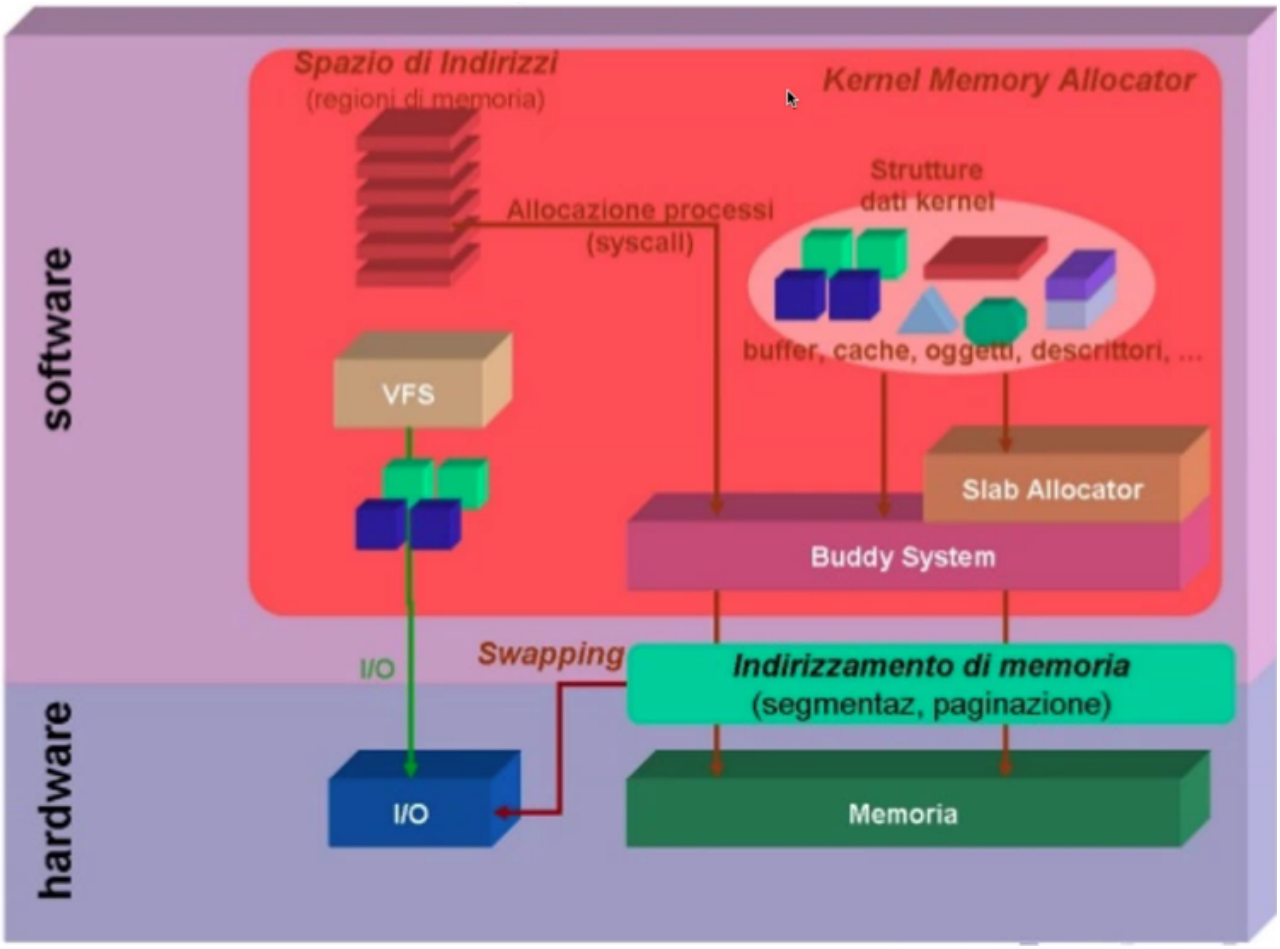
Gestisce le richieste di memoria delle applicazione traducendole in istruzioni da impartire all'unità HW di gestione della memoria

Vantaggi:

- Esecuzione concorrente dei processi;
- Esecuzione di applicazioni con richieste di memoria maggiori della memoria fisica;
- Condivisione da parte dei processi di una singola immagine di memoria di una libreria o del programma;
- Programmi rilocati



3.6 Memoria virtuale in Linux



Buddy System → tecnica di allocazione dinamica della memoria che divide la memoria in partizioni per soddisfare una richiesta di memoria nel miglior modo possibile.

Slab allocation → tecnica di allocazione della memoria per una migliore allocazione degli oggetti. Riduce la frammentazione causata dalla allocazione e della deallocazione

Kernel Memory Allocator

È la componente del kernel che cerca di soddisfare richesta di aree di memoria provenienti dal kernel stesso e dalle applicazioni

- Il KMA risponde a richieste di memoria provenienti da:
  - altri sottosistemi del kernel per uso del SO;

  - dai programmi utenti per mezzo di system call.

- Un KMA dovrebbe avere queste caratteristiche:

  - essere veloce;
  - minimizzare sprechi di memoria;
  - ridurre la frammentazione;
  - cooperare con gli altri sottosistemi di memoria per ottenere e rilasciare pagine

3.7 Spazio di indirizzi di un processo

Lo spazio di indirizzi virtuali che ogni processo può usare è diverso dall'insieme degli indirizzi fisici che usa ad ogni istante

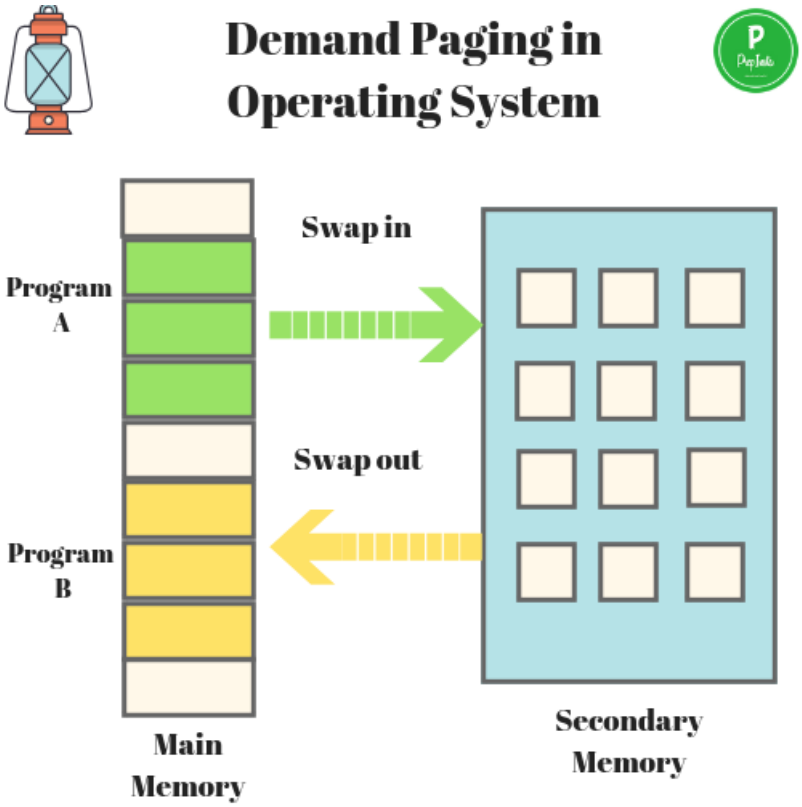
Il kernel rappresenta lo spazio di indirizzi come lista di descrittori di regioni di memoria

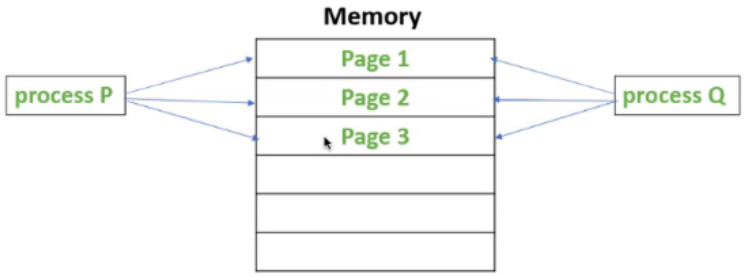
Alla creazione di un processo il kernel gli associa uno spazio di indirizzi che include regioni di memoria per:

- codice eseguibile;
- i dati inizializzati (e non) del programma;
- lo stack del programma;
- il codice eseguibile e i dati delle librerie condivise;
- lo heap (memoria dinamicamente richiesta dal processo);
- mapping in memoria di file usati dal programma

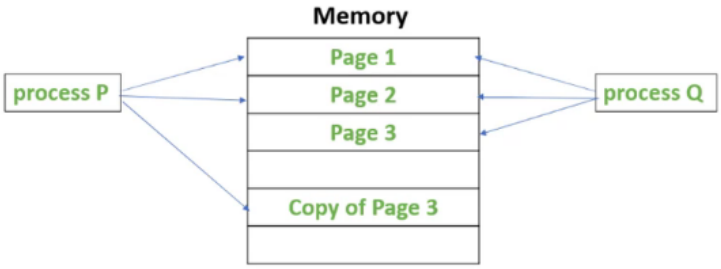
I moderni SO Unix usano strategie efficienti di allocazione di pagine basate sul Demand Paging e Copy-on-Write

Demand Pagin e Copy-on-Write a confronto

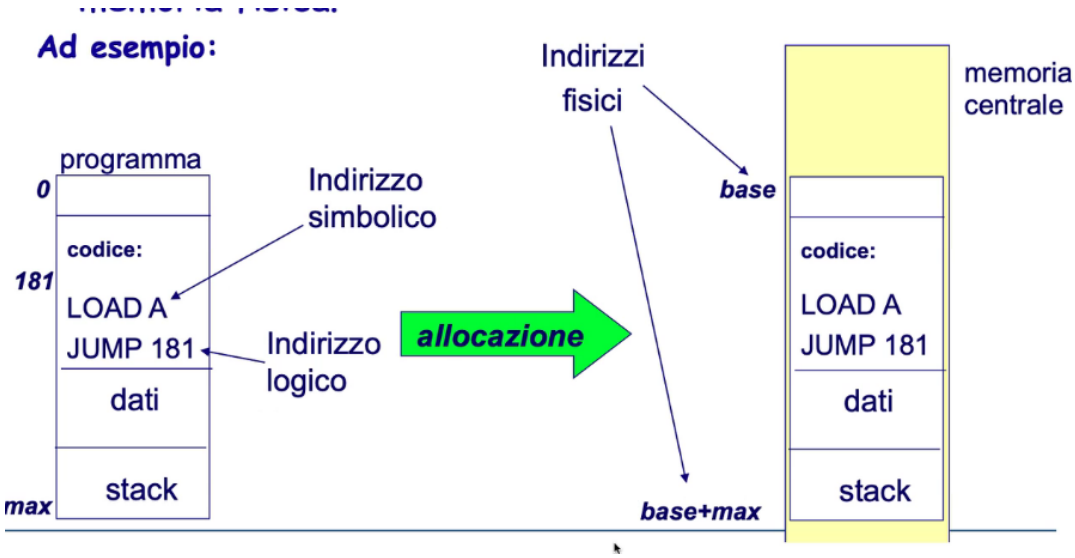




Before process P modifies Page 3



After process P modifies Page 3

Con la fork padre e figlio non lavorano sullo stesso spazio di indirizzamento, ma il figlio lavora su una copia dei dati del padre (a differenza dei thread che condividono la stessa memoria)

Ogni processo dispone di un proprio spazio di indirizzamento logico `[0,max]` che viene allocata nella memoria fisica